Solo E-Commerce Platform — Technical Features Document

Document Version: 1.0
Date: 17 March 2026
Author: Solo Engineering Team
Status: Final

Table of Contents

---

1. Authentication & User Identity

Overview

Full user lifecycle management including registration, login, token-based sessions, password management, and role-based access control.

Technical Stack

- **Password Hashing**: Argon2id (memoryCost: 65536, timeCost: 3, parallelism: 4, hashLength: 32)
- **Token Strategy**: Dual JWT (access + refresh)
- **Storage**: PostgreSQL `users` table, `refresh_tokens` table

API Endpoints

Method	Endpoint	Auth	Description
POST	`/api/auth/register`	Public	Create new account
POST	`/api/auth/login`	Public	Authenticate and receive tokens
POST	`/api/auth/refresh`	Public	Exchange refresh token for new access token
POST	`/api/auth/logout`	Bearer	Revoke refresh token
POST	`/api/auth/forgot-password`	Public	Send password reset email
POST	`/api/auth/reset-password`	Public	Reset password with token
PATCH	`/api/auth/change-password`	Bearer	Change password (requires current password)
POST	`/api/auth/verify-email`	Public	Verify email with token
POST	`/api/auth/resend-verification`	Public	Resend verification email

Token Structure

```
Access Token (JWT, 15-minute expiry):
{
  "sub": "uuid",           // User ID
  "email": "string",
  "role": "CUSTOMER|ADMIN|SUPER_ADMIN",
  "iat": 1234567890,
  "exp": 1234568790
}

Refresh Token (JWT, 7-day expiry):
{
  "sub": "uuid",
  "jti": "uuid",           // Token ID (stored in DB for revocation)
  "iat": 1234567890,
  "exp": 1235172690
}
```

Database Tables

Table	Key Columns
`users`	id (UUID PK), email (unique), password_hash, first_name, last_name, phone, role, is_email_verified, is_active, created_at
`refresh_tokens`	id (UUID PK), user_id (FK), token_hash, expires_at, is_revoked

Business Logic

1. **Registration**: Validate unique email → Hash password (Argon2id) → Create user with role CUSTOMER → Generate verification token → Send verification email
2. **Login**: Find user by email → Verify password against hash → Generate access + refresh tokens → Store refresh token hash → Return tokens
3. **Token Refresh**: Validate refresh token JWT → Check not revoked in DB → Revoke old refresh token → Issue new access + refresh pair (rotation)
4. **Password Reset**: Generate 6-character reset token → Store hashed in DB with 1-hour expiry → Send email → On submit: validate token, hash new password, update user
5. **Rate Limiting**: Login endpoint limited to 5 attempts per email per 15-minute window

Frontend Integration

- **Provider**: `AuthProvider` manages auth state (logged in/out, current user)
- **Storage**: Access token in memory; refresh token in `SharedPreferences`
- **Interceptor**: `ApiClient` automatically attaches Bearer token, auto-refreshes on 401
- **Screens**: `LoginScreen`, `RegisterScreen`, `ForgotPasswordScreen`, `ResetPasswordScreen`

---

2. Product Catalog

Overview

Core product display system powering the storefront — listing pages, category pages, and product detail views.

API Endpoints

Method	Endpoint	Auth	Description
GET	`/api/products`	Public	List products with pagination and filters
GET	`/api/products/:id`	Public	Get single product detail
GET	`/api/products/slug/:slug`	Public	Get product by URL slug
GET	`/api/products/featured`	Public	Get featured products
GET	`/api/products/best-sellers`	Public	Get best-selling products
GET	`/api/products/new-arrivals`	Public	Get newest products
GET	`/api/products/category/:categoryId`	Public	Products filtered by category
GET	`/api/products/brand/:brandId`	Public	Products filtered by brand

Query Parameters (GET /api/products)

Parameter	Type	Default	Description
`page`	number	1	Page number
`limit`	number	20	Items per page (max 100)
`categoryId`	UUID	—	Filter by category
`subcategoryId`	UUID	—	Filter by subcategory
`brandId`	UUID	—	Filter by brand
`minPrice`	number	—	Minimum selling price
`maxPrice`	number	—	Maximum selling price
`search`	string	—	Text search (name + description)
`sortBy`	string	`createdAt`	Sort field
`sortOrder`	string	`DESC`	Sort direction (ASC/DESC)

Parameter	Type	Default	Description
`isFeatured`	boolean	—	Filter featured only
`isBestSeller`	boolean	—	Filter best sellers
`isNew`	boolean	—	Filter new arrivals
`isActive`	boolean	—	Filter active only (default true for public)

Response Structure


```
{
  "data": [
    {
      "id": "uuid",
      "name": "Eva Solo Nordic Kitchen Knife 18cm",
      "slug": "eva-solo-nordic-kitchen-knife-18cm",
      "description": "...",
      "shortDescription": "...",
      "sellingPrice": 249.00,
      "originalPrice": 299.00,
      "costPrice": null,
      "currency": "AED",
      "vatRate": 5,
      "isActive": true,
      "isFeatured": true,
      "isBestSeller": false,
      "isNew": true,
      "brand": { "id": "uuid", "name": "Eva Solo" },
      "category": { "id": "uuid", "name": "Kitchenware" },
      "subcategory": { "id": "uuid", "name": "Knives" },
      "images": [
        { "id": "uuid", "url": "/uploads/abc123.webp", "displayOrder": 0 }
      ],
      "specifications": [
        { "key": "Material", "value": "Stainless Steel" }
      ],
      "dimensions": { "width": 5.0, "height": 32.0, "depth": 2.5, "unit": "cm" },
      "createdAt": "2025-01-15T10:30:00Z"
    }
  ],
  "meta": {
    "total": 805,
    "page": 1,
    "limit": 20,
    "totalPages": 41
  }
}
```

Image Resolution Pipeline

Product images are stored as UUID references in `product_images` → linked to `media_assets` table → resolved at query time by `resolveProductImageUrls()`:

1. Query products from database
2. Collect all `mediaAssetId` references from `product_images`
3. Batch-fetch `media_assets` records
4. Map UUID → `"/uploads/" + media_asset.key`
5. Attach resolved URLs to each product's `images` array

Database Tables

Table	Key Columns
`products`	id, name, slug, description, selling_price, original_price, cost_price, currency, vat_rate, brand_id, category_id, subcategory_id, is_active, is_featured, is_best_seller, is_new
`product_images`	id, product_id (FK), media_asset_id (FK), display_order
`product_specifications`	id, product_id (FK), key, value, display_order
`categories`	id, name, slug, image, description, display_order
`subcategories`	id, name, slug, category_id (FK), display_order
`brands`	id, name, slug, logo_url, description

Frontend Screens

- **ProductListingScreen**: Grid/list of products with filter panel (category, brand, price)
- **ProductDetailScreen**: Full product info with image gallery, specs, add-to-cart, favorite toggle
- **CategoryScreen**: Products for a specific category with subcategory tabs
- **BrandScreen**: Products for a specific brand

Frontend Providers

- **ProductProvider**: Fetches and caches product lists, manages pagination state
- **ProductDetailProvider**: Fetches single product details, manages loading state

---

3. Product Search & Filtering

Overview

Text-based search with combinable filters for category, brand, and price range.

Technical Implementation

Backend Search Query (Prisma):

```
WHERE (
  name ILIKE '%search_term%'
  OR description ILIKE '%search_term%'
)
AND category_id = :categoryId (if provided)
AND brand_id = :brandId (if provided)
AND selling_price >= :minPrice (if provided)
AND selling_price <= :maxPrice (if provided)
AND is_active = true
```

API Endpoint

- `GET /api/products?search=nordic+kitchen&categoryId=xxx&minPrice=100&maxPrice=500`

Frontend Components

- **SearchBar widget**: Header search input, triggers search on submit
- **FilterPanel widget**: Category dropdown, brand dropdown, price range slider
- **SearchResultsScreen**: Displays matches in product grid with count

Behavior

- Search is case-insensitive (PostgreSQL ILIKE)
- Filters combine with AND logic
- Empty search returns all products (with applied filters)
- Results are paginated (20 per page)
- Minimum 1-character search term required

---

4. Shopping Cart

Overview

Server-persisted shopping cart allowing logged-in users to add, update, and remove items with real-time price calculation.

API Endpoints

Method	Endpoint	Auth	Description
GET	`/api/cart`	Bearer	Get current user's cart with enriched items

Method	Endpoint	Auth	Description
POST	`/api/cart/items`	Bearer	Add item to cart
PATCH	`/api/cart/items/:itemId`	Bearer	Update item quantity
DELETE	`/api/cart/items/:itemId`	Bearer	Remove item from cart
DELETE	`/api/cart`	Bearer	Clear entire cart
POST	`/api/cart/apply-promo`	Bearer	Apply promo code to cart
DELETE	`/api/cart/remove-promo`	Bearer	Remove applied promo code

Cart Enrichment Flow

When GET `/api/cart` is called:

1. Fetch cart record for authenticated user
2. Fetch all cart items with product references
3. For each item:
    - Resolve current product price (protects against stale prices)
    - Resolve product images via `resolveProductImageUrls()`
    - Calculate line total: `price × quantity`
4. Calculate cart totals:
    - Subtotal = Σ (lineTotal)
    - Discount = apply promo code rules (if any)
    - VAT = (subtotal - discount) × vatRate / 100
    - Total = subtotal - discount + VAT

Response Structure

```
{
  "id": "uuid",
  "userId": "uuid",
  "items": [
    {
      "id": "uuid",
      "productId": "uuid",
      "productName": "Eva Solo Nordic Kitchen Knife",
      "productImage": "/uploads/abc123.webp",
      "unitPrice": 249.00,
      "quantity": 2,
      "lineTotal": 498.00
    }
  ],
  "promoCode": null,
  "subtotal": 498.00,
  "discount": 0.00,
  "vatRate": 5,
  "vatAmount": 24.90,
  "total": 522.90,
  "itemCount": 2
}
```

Database Tables

Table	Key Columns
`carts`	id (UUID PK), user_id (FK, unique), promo_code_id (FK, nullable), created_at, updated_at
`cart_items`	id (UUID PK), cart_id (FK), product_id (FK), quantity, created_at

Business Rules

- One cart per user (created lazily on first add)
- Quantity minimum: 1 (lower = remove)
- Quantity maximum: 99
- Adding existing product increments quantity
- Cart cleared after successful order placement
- Product prices always read fresh from `products` table (not cached in cart)

Frontend Integration

- **CartProvider**: Manages cart state, item count badge, total calculation
- **CartScreen**: Full cart view with quantity controls, promo input, totals
- **CartIcon widget**: Header cart icon with item count badge
- **API Service**: `CartApiService` with all CRUD methods

---

5. Checkout & Payment

Overview

Multi-step checkout flow with address selection, Stripe payment, and order creation.

Checkout Flow Sequence

```
Step 1: SHIPPING ADDRESS
   ├── Select existing address -OR- add new address
   └── Validate address fields

Step 2: BILLING ADDRESS
   ├── Same as shipping (checkbox)
   └── Select existing -OR- add new

Step 3: PAYMENT
   ├── Create Stripe PaymentIntent (server-side)
   ├── Display Stripe card element
   └── Confirm payment (client-side)

Step 4: CONFIRMATION
   ├── Display order summary
   └── Show order number
```

API Endpoints

Method	Endpoint	Auth	Description
POST	`/api/orders`	Bearer	Create order (triggers payment)
POST	`/api/payments/create-intent`	Bearer	Create Stripe PaymentIntent
POST	`/api/payments/confirm`	Bearer	Confirm Stripe payment

Payment Flow (Stripe)

1. **Frontend** → POST `/api/payments/create-intent` with cart total
2. **Backend** → `stripe.paymentIntents.create({ amount, currency: 'aed' })` → return `clientSecret`
3. **Frontend** → `stripe.confirmCardPayment(clientSecret, { payment_method: { card } })`
4. **Stripe** → processes payment, returns result
5. **Frontend** → POST `/api/orders` with `paymentIntentId`, `shippingAddressId`, `billingAddressId`
6. **Backend** → Validates payment status → Creates order → Clears cart → Awards loyalty points → Sends confirmation email

Order Creation Logic (12 Steps)

1. Validate cart is not empty
2. Validate shipping and billing addresses belong to user
3. Verify Stripe PaymentIntent status = 'succeeded'
4. Fetch current product prices (prevent stale pricing)
5. Calculate subtotal from current prices
6. Apply promo code discount (if applicable)
7. Calculate VAT
8. Calculate grand total
9. Snapshot addresses as JSON (immutable records)
10. Create order record + order items
11. Clear user's cart
12. Award loyalty points (1 point per AED spent)

Database Tables

Table	Key Columns
`orders`	id, order_number, user_id, status, subtotal, discount, vat_amount, shipping_amount, total, payment_intent_id, shipping_address (JSON), billing_address (JSON), promo_code_id, notes
`order_items`	id, order_id, product_id, product_name, product_image, unit_price, quantity, line_total
`order_status_history`	id, order_id, status, changed_by, notes, created_at

Frontend Integration

- **CheckoutProvider**: Manages multi-step state, address selection, payment
- **CheckoutScreen**: Step indicator, address forms, payment form, review
- **AddressFormWidget**: Reusable address input form
- **StripePaymentWidget**: Stripe card element integration

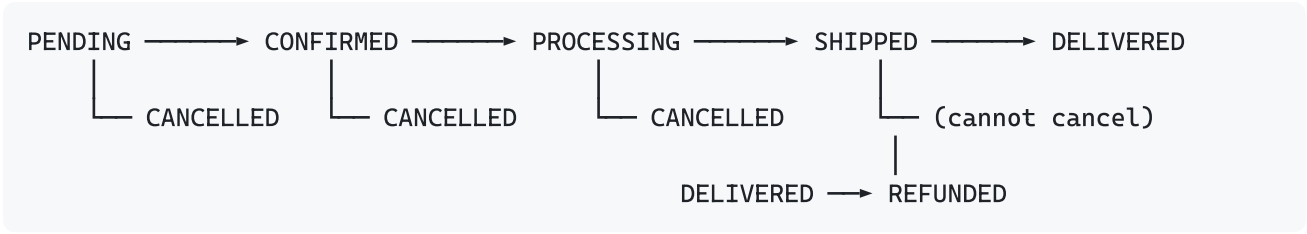
---

6. Order Management

Overview

Full order lifecycle from creation through delivery, with status tracking and history.

Order Status State Machine



Valid Status Transitions

From	Allowed To
PENDING	CONFIRMED, CANCELLED
CONFIRMED	PROCESSING, CANCELLED
PROCESSING	SHIPPED, CANCELLED
SHIPPED	DELIVERED
DELIVERED	REFUNDED

API Endpoints (Customer)

Method	Endpoint	Auth	Description
GET	`/api/orders`	Bearer	List user's orders
GET	`/api/orders/:id`	Bearer	Get order detail (own orders only)
GET	`/api/orders/:id/invoice`	Bearer	Download invoice PDF

API Endpoints (Admin)

Method	Endpoint	Auth	Description
GET	`/api/admin/orders`	Admin	List all orders with filters
GET	`/api/admin/orders/:id`	Admin	Get any order detail
PATCH	`/api/admin/orders/:id/status`	Admin	Update order status
GET	`/api/admin/orders/:id/invoice`	Admin	Generate/download invoice

Status Update Payload

```
{
  "status": "SHIPPED",
  "trackingNumber": "ARAMEX-12345678",
  "notes": "Shipped via Aramex standard delivery"
}
```

Business Logic

- Status transitions are validated against the state machine (invalid transitions return 400)
- Each status change creates an entry in `order_status_history` with timestamp and actor
- Customers can only view their own orders (enforced by user_id filter)
- Admin can view all orders and filter by status, date range, customer

Frontend Screens

- **OrderHistoryScreen** (customer): List of past orders with status badges
- **OrderDetailScreen** (customer): Full order with items, addresses, timeline
- **AdminOrderListScreen**: Sortable/filterable order table
- **AdminOrderDetailScreen**: Order detail with status update controls

---

7. Invoice Generation

Overview

Server-side PDF invoice generation for completed orders.

Technical Implementation

- **Library**: PDFKit (Node.js)
- **Endpoint**: `GET /api/orders/:id/invoice`
- **Auth**: Bearer token (customer — own orders) or Admin role
- **Response**: Binary PDF stream with `Content-Type: application/pdf`

Invoice Content

1. **Header**: Company logo, company name, company address
2. **Order Info**: Order number, order date, payment method
3. **Customer Info**: Name, email, shipping address, billing address
4. **Items Table**: Product name, quantity, unit price, line total
5. **Totals**: Subtotal, discount (if any), VAT (5%), grand total
6. **Footer**: Terms and conditions, return policy

Business Rules

- Invoices are generated on-demand (not stored)
- Invoice number = order number
- Prices reflect order-time snapshot (from `order_items`, not current product prices)
- VAT amount explicitly shown as line item
- Currency shown as AED

---

8. Favorites / Wishlist

Overview

Allows customers to save products to a favorites list for quick access later.

API Endpoints

Method	Endpoint	Auth	Description
GET	`/api/favorites`	Bearer	Get user's favorites list
POST	`/api/favorites/:productId`	Bearer	Add product to favorites
DELETE	`/api/favorites/:productId`	Bearer	Remove product from favorites

Database Table

Table	Key Columns
`favorites`	id (UUID PK), user_id (FK), product_id (FK), created_at
	UNIQUE constraint on (user_id, product_id)

Technical Behavior

- **Toggle pattern**: POST if not favorited, DELETE if already favorited
- **Idempotent**: Adding an already-favorited product returns success (no duplicate error)
- **Response**: GET returns full product objects (enriched with images) in favorites list
- **Product cards**: Heart icon toggles filled/unfilled based on favorite status

Frontend Integration

- **FavoritesProvider**: Tracks favorite product IDs, exposes `isFavorite(productId)` and `toggle(productId)`
- **FavoritesScreen**: Grid of favorited products
- **HeartIcon widget**: Animated toggle on product cards and detail page
- **API Service**: `FavoritesApiService`

---

9. Loyalty Points Program

Overview

Rewards program where customers earn points on purchases and redeem them as discounts.

Points Rules

- **Earning**: 1 point per AED 1 spent (rounded down)
- **Redemption rate**: 100 points = AED 1 discount
- **Minimum redemption**: 100 points
- **Maximum redemption**: Cannot exceed order total
- **Earning trigger**: Points awarded on order creation

API Endpoints

Method	Endpoint	Auth	Description
GET	`/api/loyalty/balance`	Bearer	Get current points balance
GET	`/api/loyalty/history`	Bearer	Get points transaction history
POST	`/api/loyalty/redeem`	Bearer	Redeem points on current cart
POST	`/api/admin/loyalty/adjust`	Admin	Adjust customer points (credit/debit)

Database Table

Table	Key Columns
`loyalty_transactions`	id, user_id, points (positive=earn, negative=redeem), type (EARNED/REDEEMED/ADJUSTED), order_id (nullable), description, created_at

Balance Calculation

```
SELECT SUM(points) FROM loyalty_transactions WHERE user_id = :userId
```

Business Logic

1. After order creation: INSERT loyalty_transactions (type=EARNED, points=floor(order_total))
2. On redemption: Check balance >= requested → INSERT (type=REDEEMED, points=-requested) → Apply to cart as discount
3. Admin adjustment: INSERT (type=ADJUSTED, points=±amount, description=reason)

Frontend Integration

- **LoyaltyProvider**: Fetches and tracks balance
- **ProfileScreen**: Displays points balance and history
- **CartScreen**: Redeem points option during checkout

---

10. Promotional Codes

Overview

Discount codes that customers can apply to their cart during checkout.

Promo Code Types

Type	Behavior	Example
`PERCENTAGE`	Discount = (subtotal × percentage / 100)	10% off
`FIXED_AMOUNT`	Discount = fixed AED amount	AED 50 off
`FREE_SHIPPING`	Shipping fee set to 0	Free delivery

API Endpoints

Method	Endpoint	Auth	Description
POST	`/api/cart/apply-promo`	Bearer	Validate and apply promo to cart
DELETE	`/api/cart/remove-promo`	Bearer	Remove promo from cart
GET	`/api/admin/promo-codes`	Admin	List all promo codes
POST	`/api/admin/promo-codes`	Admin	Create promo code
PATCH	`/api/admin/promo-codes/:id`	Admin	Update promo code
DELETE	`/api/admin/promo-codes/:id`	Admin	Delete promo code

Validation Checks (6-step)

1. **Exists**: Code exists in database
2. **Active**: `is_active = true`
3. **Date range**: `start_date <= now() <= end_date`
4. **Usage limit**: `current_usage < max_usage` (total)
5. **Per-user limit**: User hasn't exceeded `max_per_user` uses
6. **Minimum order**: Cart subtotal >= `min_order_amount`

Database Table

Table	Key Columns
`promo_codes`	id, code (unique), type (PERCENTAGE/FIXED_AMOUNT/FREE_SHIPPING), value, min_order_amount, max_usage, max_per_user, current_usage, start_date, end_date, is_active

Discount Calculation

```
if type == PERCENTAGE:
    discount = subtotal × (value / 100)
    if discount > max_discount:    // optional cap
        discount = max_discount
elif type == FIXED_AMOUNT:
    discount = min(value, subtotal)   // can't exceed subtotal
elif type == FREE_SHIPPING:
    shipping = 0
```

Frontend Integration

- **CartScreen**: Promo code input field with "Apply" button
- **Validation feedback**: Success message with discount amount, or error message
- **AdminPromoScreen**: CRUD form for managing promo codes

---

11. User Profile & Address Management

Overview

Account settings and multi-address management for customers.

API Endpoints

Method	Endpoint	Auth	Description
GET	`/api/users/profile`	Bearer	Get current user profile
PATCH	`/api/users/profile`	Bearer	Update profile fields
GET	`/api/addresses`	Bearer	List user's addresses
POST	`/api/addresses`	Bearer	Create new address
PATCH	`/api/addresses/:id`	Bearer	Update address
DELETE	`/api/addresses/:id`	Bearer	Delete address
PATCH	`/api/addresses/:id/default`	Bearer	Set as default address

Profile Fields

Field	Editable	Validation
email	No	Read-only after registration
firstName	Yes	Required, max 100 chars
lastName	Yes	Required, max 100 chars
phone	Yes	Optional, valid phone format

Address Fields

Field	Required	Description
label	No	Friendly name ("Home", "Office")
firstName	Yes	Recipient first name
lastName	Yes	Recipient last name
addressLine1	Yes	Street address
addressLine2	No	Apartment, suite, etc.
city	Yes	City name
state	No	State/emirate
postalCode	No	Postal/ZIP code
country	Yes	Country (default: UAE)
phone	No	Contact number
isDefault	No	Boolean flag

Database Table

Table	Key Columns
`addresses`	id, user_id (FK), label, first_name, last_name, address_line1, address_line2, city, state, postal_code, country, phone, is_default

Business Rules

- Only one address can be `isDefault = true` per user
- Setting a new default automatically un-defaults the previous one
- Address deletion is soft (addresses used in orders are preserved as JSON snapshots)
- Customers can only manage their own addresses (enforced by user_id)

Frontend Integration

- **ProfileProvider**: Manages user profile state
- **ProfileScreen**: Edit profile form
- **AddressListScreen**: List of saved addresses with edit/delete/default controls
- **AddressFormWidget**: Reusable address form (used in both profile and checkout)

---

12. Admin Dashboard & Analytics

Overview

Administrative dashboard providing real-time business metrics and KPIs.

API Endpoints

Method	Endpoint	Auth	Description
GET	`/api/admin/dashboard/stats`	Admin	Aggregate statistics
GET	`/api/admin/dashboard/revenue-chart`	Admin	Revenue time series
GET	`/api/admin/dashboard/recent-orders`	Admin	Latest 10 orders
GET	`/api/admin/dashboard/top-products`	Admin	Top selling products
GET	`/api/admin/dashboard/order-status-distribution`	Admin	Order count by status

Dashboard Stats Response

```
{
  "totalRevenue": 125430.50,
  "totalOrders": 342,
  "newCustomers": 45,
  "averageOrderValue": 366.76,
  "period": "current_month",
  "comparison": {
    "revenueChange": 12.5,
    "ordersChange": 8.3,
    "customersChange": 15.2
  }
}
```

Queries Used

```
-- Total revenue (current month)
SELECT SUM(total) FROM orders
WHERE created_at >= date_trunc('month', NOW())
AND status NOT IN ('CANCELLED', 'REFUNDED');

-- New customers (current month)
SELECT COUNT(*) FROM users
WHERE role = 'CUSTOMER'
AND created_at >= date_trunc('month', NOW());

-- Top selling products
SELECT p.name, SUM(oi.quantity) as sold, SUM(oi.line_total) as revenue
FROM order_items oi
JOIN products p ON oi.product_id = p.id
GROUP BY p.id ORDER BY sold DESC LIMIT 10;
```

Frontend Screen

- **AdminDashboardScreen**: Card grid (revenue, orders, customers, AOV) + charts + recent orders table + top products table

---

13. Admin Product Management

Overview

Full CRUD interface for managing the product catalog.

API Endpoints

Method	Endpoint	Auth	Description
GET	`/api/admin/products`	Admin	List all products (incl. inactive)
GET	`/api/admin/products/:id`	Admin	Get product detail with all fields
POST	`/api/admin/products`	Admin	Create new product
PATCH	`/api/admin/products/:id`	Admin	Update product fields
DELETE	`/api/admin/products/:id`	Admin	Delete product
POST	`/api/admin/products/:id/images`	Admin	Upload product images
DELETE	`/api/admin/products/:id/images/:imageId`	Admin	Remove product image
PATCH	`/api/admin/products/:id/images/reorder`	Admin	Reorder images

Create/Update Product DTO

```
{
  "name": "string (required)",
  "slug": "string (auto-generated if omitted)",
  "description": "string",
  "shortDescription": "string",
  "sellingPrice": "number (required)",
  "originalPrice": "number",
  "costPrice": "number",
  "currency": "AED",
  "vatRate": 5,
  "categoryId": "uuid (required)",
  "subcategoryId": "uuid",
  "brandId": "uuid (required)",
  "designerId": "uuid",
  "countryId": "uuid",
  "isActive": true,
  "isFeatured": false,
  "isBestSeller": false,
  "isNew": false,
  "specifications": [
    { "key": "Material", "value": "Stainless Steel" }
  ],
  "dimensions": {
    "width": 5.0,
    "height": 32.0,
    "depth": 2.5,
    "unit": "cm"
  }
}
```

Business Rules

- Slug auto-generated from name if not provided (kebab-case, unique)
- `costPrice` is admin-only (never exposed to customers)
- Deleting a product used in past orders: product remains referenced in `order_items` (snapshot data)
- Image upload max: 5 MB per file, formats: JPEG, PNG, WebP, SVG
- First image (display_order = 0) is the primary/thumbnail image

Frontend Screens

- **AdminProductListScreen**: Paginated table with search, filter by category/brand/status
- **AdminProductFormScreen**: Create/edit form with all fields, image upload/drag-reorder

---

14. Admin Category & Brand Management

Overview

Management of product taxonomy (categories, subcategories) and brands.

Category API Endpoints

Method	Endpoint	Auth	Description
GET	`/api/categories`	Public	List all categories
GET	`/api/categories/:id`	Public	Get category with subcategories
POST	`/api/admin/categories`	Admin	Create category
PATCH	`/api/admin/categories/:id`	Admin	Update category
DELETE	`/api/admin/categories/:id`	Admin	Delete category
POST	`/api/admin/categories/:id/subcategories`	Admin	Create subcategory
PATCH	`/api/admin/subcategories/:id`	Admin	Update subcategory
DELETE	`/api/admin/subcategories/:id`	Admin	Delete subcategory

Brand API Endpoints

Method	Endpoint	Auth	Description
GET	`/api/brands`	Public	List all brands
GET	`/api/brands/:id`	Public	Get brand detail
POST	`/api/admin/brands`	Admin	Create brand
PATCH	`/api/admin/brands/:id`	Admin	Update brand
DELETE	`/api/admin/brands/:id`	Admin	Delete brand

Current Data

- **Categories**: Kitchenware, Tableware & Glassware, Home (3 categories)
- **Brands**: Eva Solo, Eva Trio, PWtbS, Eva (4 brands)

Business Rules

- Categories and brands cannot be deleted if products reference them

- Subcategories belong to exactly one category
- Display order determines menu sequence
- Slugs are auto-generated and unique per type

---

15. Admin Customer Management

Overview

Admin tools for viewing and managing customer accounts, addresses, and loyalty points.

API Endpoints

Method	Endpoint	Auth	Description
GET	`/api/admin/customers`	Admin	List all customers (paginated, searchable)
GET	`/api/admin/customers/:id`	Admin	Get customer detail + order history
POST	`/api/admin/customers`	Admin	Create customer account
PATCH	`/api/admin/customers/:id`	Admin	Update customer profile
DELETE	`/api/admin/customers/:id`	Admin	Soft-delete customer
GET	`/api/admin/customers/:id/addresses`	Admin	Get customer addresses
POST	`/api/admin/customers/:id/addresses`	Admin	Add address for customer
PATCH	`/api/admin/customers/:id/addresses/:addressId`	Admin	Update customer address
DELETE	`/api/admin/customers/:id/addresses/:addressId`	Admin	Delete customer address
POST	`/api/admin/customers/:id/loyalty/adjust`	Admin	Adjust loyalty points

Search/Filter Options

- Search by name, email
- Filter by: active/inactive, email verification status, registration date range
- Sort by: name, email, created date, total orders, total spend

Frontend Screens

- **AdminCustomerListScreen**: Searchable customer table
- **AdminCustomerDetailScreen**: Profile info, order history, addresses, loyalty balance

---

16. Admin Order Management

Overview

Order processing interface for admins to track, update, and manage all orders.

API Endpoints

(See Section 6 - Order Management for full endpoint list)

Admin-specific Features

1. **Bulk status view**: Filter orders by status (PENDING, CONFIRMED, PROCESSING, SHIPPED, DELIVERED, CANCELLED, REFUNDED)
2. **Status update**: Change status with validation against state machine
3. **Add tracking**: Attach tracking number when marking as SHIPPED
4. **Order notes**: Internal notes visible only to admin
5. **Invoice generation**: Generate and download customer invoice PDF
6. **Order timeline**: Full history of all status changes with timestamps and actor

Frontend Screens

- **AdminOrderListScreen**: Filterable, sortable order table with status badges and color coding
- **AdminOrderDetailScreen**: Full order detail with action buttons for status transitions, tracking input, notes

---

17. Banner & CMS Management

Overview

Content management system for homepage hero banners, promotional banners, and section configuration.

API Endpoints

Method	Endpoint	Auth	Description
GET	`/api/banners`	Public	Get active banners by placement

Method	Endpoint	Auth	Description
GET	`/api/banners/:id`	Public	Get single banner
GET	`/api/admin/banners`	Admin	List all banners
POST	`/api/admin/banners`	Admin	Create banner
PATCH	`/api/admin/banners/:id`	Admin	Update banner
DELETE	`/api/admin/banners/:id`	Admin	Delete banner
GET	`/api/homepage`	Public	Get composed homepage data
GET	`/api/admin/homepage/sections`	Admin	List all homepage sections
PATCH	`/api/admin/homepage/sections/:id`	Admin	Update section config

Banner Model

```
{
  "id": "uuid",
  "title": "Summer Collection 2025",
  "subtitle": "Up to 30% off premium kitchenware",
  "imageUrl": "/uploads/banner-hero.webp",
  "mobileImageUrl": "/uploads/banner-hero-mobile.webp",
  "ctaText": "Shop Now",
  "ctaUrl": "/category/kitchenware",
  "placement": "hero",
  "displayOrder": 0,
  "isActive": true,
  "startDate": "2025-06-01T00:00:00Z",
  "endDate": "2025-08-31T23:59:59Z",
  "backgroundColor": "#F5F5F5",
  "textColor": "#1A1A1A"
}
```

Banner Placements

Placement	Location	Typical Use
`hero`	Top of homepage, full-width carousel	Brand/seasonal campaigns
`top`	Below hero section	Featured collection highlight
`mid`	Middle of homepage	Cross-sell promotion
`bottom`	Near footer	Secondary promotions

Homepage Composition

The `GET /api/homepage` endpoint returns composed homepage data:

```
{
  "heroBanners": [...],
  "featuredProducts": [...],
  "bestSellers": [...],
  "newArrivals": [...],
  "categoryTiles": [...],
  "promotionalBanners": [...],
  "sections": [
    { "type": "hero", "order": 0, "isActive": true },
    { "type": "featured", "order": 1, "isActive": true },
    { "type": "categories", "order": 2, "isActive": true },
    ...
  ]
}
```

Database Tables

Table	Key Columns
`banners`	id, title, subtitle, image_url, mobile_image_url, cta_text, cta_url, placement, display_order, is_active, start_date, end_date, background_color, text_color
`homepage_sections`	id, type, display_order, is_active, config (JSON)

CTA URL Validation

- `ctaUrl` validated to ensure:
  - Must be a relative path starting with `/` (no external URLs)
  - No JavaScript or data URIs (XSS prevention)
  - Max length 500 characters

Frontend Screens

- **HomeScreen**: Renders homepage sections in configured order
- **AdminBannerListScreen**: List/manage all banners
- **AdminBannerFormScreen**: Create/edit banner with image upload and preview
- **AdminHomepageSectionsScreen**: Drag-reorder and toggle sections

---

18. Landing Pages

Overview

Admin-configurable landing pages for categories, promotions, or custom content.

API Endpoints

Method	Endpoint	Auth	Description
GET	`/api/landing-pages/:slug`	Public	Get landing page by slug
GET	`/api/admin/landing-pages`	Admin	List all landing pages
POST	`/api/admin/landing-pages`	Admin	Create landing page
PATCH	`/api/admin/landing-pages/:id`	Admin	Update landing page
DELETE	`/api/admin/landing-pages/:id`	Admin	Delete landing page

Landing Page Structure

```
{
  "id": "uuid",
  "title": "Summer Sale",
  "slug": "summer-sale",
  "metaTitle": "Summer Sale - Solo E-Commerce",
  "metaDescription": "...",
  "sections": [
    {
      "type": "banner",
      "content": { "imageUrl": "...", "title": "..." }
    },
    {
      "type": "product_grid",
      "content": { "productIds": ["uuid", "uuid"] }
    },
    {
      "type": "text",
      "content": { "body": "..." }
    }
  ],
  "isActive": true
}
```

Database Table

Table	Key Columns
`landing_pages`	id, title, slug (unique), meta_title, meta_description, sections (JSON), is_active, created_at, updated_at

---

19. Blog System

Overview

Content marketing blog with categories and tags (backend complete, frontend in progress).

API Endpoints

Method	Endpoint	Auth	Description
GET	`/api/blog/posts`	Public	List published posts (paginated)
GET	`/api/blog/posts/:slug`	Public	Get single post
GET	`/api/blog/categories`	Public	List blog categories
GET	`/api/admin/blog/posts`	Admin	List all posts (incl. drafts)
POST	`/api/admin/blog/posts`	Admin	Create post
PATCH	`/api/admin/blog/posts/:id`	Admin	Update post
DELETE	`/api/admin/blog/posts/:id`	Admin	Delete post

Blog Post Model

```
{
  "id": "uuid",
  "title": "...",
  "slug": "...",
  "excerpt": "...",
  "content": "...",
  "featuredImage": "/uploads/blog-post-1.webp",
  "author": { "id": "...", "name": "..." },
  "category": { "id": "...", "name": "..." },
  "tags": ["tag1", "tag2"],
  "status": "PUBLISHED",
  "publishedAt": "2025-03-15T10:00:00Z",
  "createdAt": "..."
}
```

Database Tables

Table	Key Columns
`blog_posts`	id, title, slug, excerpt, content, featured_image, author_id, category_id, status (DRAFT/PUBLISHED), published_at
`blog_categories`	id, name, slug
`blog_tags`	id, name, slug

Table	Key Columns
`blog_post_tags`	post_id, tag_id (junction table)

---

20. Navigation Management

Overview

Admin-configurable navigation menus for the storefront header and footer.

API Endpoints

Method	Endpoint	Auth	Description
GET	`/api/navigation`	Public	Get active navigation menus
GET	`/api/admin/navigation`	Admin	List all nav items
POST	`/api/admin/navigation`	Admin	Create nav item
PATCH	`/api/admin/navigation/:id`	Admin	Update nav item
DELETE	`/api/admin/navigation/:id`	Admin	Delete nav item

Navigation Item Model

```
{
  "id": "uuid",
  "label": "Kitchenware",
  "url": "/category/kitchenware",
  "parentId": null,
  "displayOrder": 1,
  "isActive": true,
  "children": [
    {
      "id": "uuid",
      "label": "Knives",
      "url": "/category/kitchenware/knives",
      "parentId": "uuid",
      "displayOrder": 0
    }
  ]
}
```

Features

- Hierarchical menu support (parent → children)
- Admin drag-reorder capability
- Active/inactive toggle per item
- URL validation (relative paths only)

---

21. Media Management & Image Pipeline

Overview

Image upload, optimization, storage, and serving system.

API Endpoints

Method	Endpoint	Auth	Description
POST	`/api/media/upload`	Admin	Upload image file
GET	`/api/media`	Admin	List uploaded media
DELETE	`/api/media/:id`	Admin	Delete media file
GET	`/uploads/:key`	Public	Serve uploaded file (static)

Upload Pipeline

```
Client uploads file
    ↓
Multer receives file (max 5 MB)
    ↓
Validate file type (JPEG, PNG, WebP, SVG)
    ↓
Generate unique key: UUID + extension
    ↓
Save to /uploads/ directory
    ↓
Create media_assets record in DB
    ↓
Return { id, key, url, mimeType, size }
```

Database Table

Table	Key Columns
`media_assets`	id (UUID PK), key (unique filename), original_name, mime_type, size_bytes, uploaded_by, created_at

Image Serving

- Static files served from `/uploads/` directory via NestJS `ServeStaticModule`
- URLs constructed as `/uploads/{key}` (e.g., `/uploads/a1b2c3d4.webp`)
- Product image resolution: `product_images.media_asset_id` → `media_assets.key` → URL

Business Rules

- Accepted MIME types: `image/jpeg`, `image/png`, `image/webp`, `image/svg+xml`
- Maximum file size: 5 MB (enforced by Multer)
- Files persist on disk; deletion removes DB record and file
- Unique key prevents filename collisions

---

22. Reporting Engine

Overview

Comprehensive reporting system providing business analytics for admin users.

API Endpoints

Method	Endpoint	Auth	Description
GET	`/api/admin/reports/revenue`	Admin	Revenue report with date range
GET	`/api/admin/reports/orders`	Admin	Order statistics report
GET	`/api/admin/reports/products`	Admin	Product performance report
GET	`/api/admin/reports/customers`	Admin	Customer analytics report
GET	`/api/admin/reports/vat`	Admin	VAT collection report
GET	`/api/admin/reports/categories`	Admin	Revenue by category
GET	`/api/admin/reports/promo-codes`	Admin	Promo code usage stats
GET	`/api/admin/reports/stock`	Admin	Inventory status

Query Parameters (Common)

Param	Type	Description
`startDate`	ISO date	Report period start
`endDate`	ISO date	Report period end

Param	Type	Description
`groupBy`	string	`day`, `week`, `month`

Revenue Report Response

```
{
  "totalRevenue": 125430.50,
  "totalOrders": 342,
  "averageOrderValue": 366.76,
  "timeSeries": [
    { "date": "2025-03-01", "revenue": 8540.25, "orders": 23 },
    { "date": "2025-03-02", "revenue": 5210.75, "orders": 15 }
  ],
  "topCategories": [
    { "name": "Kitchenware", "revenue": 75000, "percentage": 59.8 }
  ],
  "topBrands": [
    { "name": "Eva Solo", "revenue": 55000, "percentage": 43.8 }
  ]
}
```

VAT Report Response

```
{
  "taxableAmount": 119457.62,
  "vatCollected": 5972.88,
  "vatRate": 5,
  "orderCount": 342,
  "period": { "start": "2025-03-01", "end": "2025-03-31" }
}
```

Frontend Screens

- **AdminReportsScreen**: Report type selector with date range picker
- Charts rendered with FL Chart library (Flutter)

---

23. VAT Configuration

Overview

System-wide VAT rate configuration for tax calculation on all orders.

API Endpoints

Method	Endpoint	Auth	Description
GET	`/api/admin/settings/vat`	Admin	Get current VAT rate
PATCH	`/api/admin/settings/vat`	Admin	Update VAT rate

VAT Calculation Logic

```
vatRate = systemSetting.vatRate   // default: 5 (%)
subtotal = Σ (item.unitPrice × item.quantity)
discount = calculatePromoDiscount(subtotal, promoCode)
taxableAmount = subtotal − discount
vatAmount = taxableAmount × (vatRate / 100)
total = taxableAmount + vatAmount + shippingAmount
```

Key Behaviors

- VAT rate is stored as a system setting (single row)
- Changes apply to future orders only (past order VAT is immutable)
- VAT amount is stored in `orders.vat_amount` at order creation time
- Product-level `vat_rate` can override system default (per-product VAT)

---

24. Stripe Configuration

Overview

Admin-configurable Stripe payment gateway settings.

API Endpoints

Method	Endpoint	Auth	Description
GET	`/api/admin/settings/stripe`	Admin	Get Stripe config (masked keys)
PATCH	`/api/admin/settings/stripe`	Admin	Update Stripe settings

Configuration Fields

Field	Description	Storage
`publishableKey`	Stripe publishable key (pk_*)	Database
`secretKey`	Stripe secret key (sk_*)	Database (encrypted)
`webhookSecret`	Webhook signing secret	Database (encrypted)

Field	Description	Storage
`isEnabled`	Payment toggle	Database
`currency`	Default currency (AED)	Database

Security

- Secret key never returned in GET response (masked: `sk_****...last4`)
- Keys stored encrypted at rest
- Only SUPER_ADMIN can modify Stripe settings
- Test mode vs live mode determined by key prefix (`sk_test_` vs `sk_live_`)

Frontend Screen

- **AdminStripeSettingsScreen**: Form to update keys and toggle payments

---

25. Email Notifications

Overview

Transactional email system for critical user communications.

Email Types

Trigger	Recipient	Template	Content
User registration	Customer	`email-verification`	Verification link
Password reset request	Customer	`password-reset`	Reset code/link
Order placed	Customer	`order-confirmation`	Order summary, items, total
Order shipped	Customer	`order-shipped`	Tracking number

Technical Implementation

- **Transport**: SMTP (configurable via environment variables)
- **Library**: `@nestjs-modules/mailer` with Handlebars templates
- **Configuration**: `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `SMTP_FROM`

Email Templates Location

```
backend/src/mail/templates/
├── email-verification.hbs
├── password-reset.hbs
├── order-confirmation.hbs
└── order-shipped.hbs
```

Business Rules

- Email failures are logged but do not block the triggering operation
- Verification tokens expire after 24 hours
- Password reset tokens expire after 1 hour
- From address: configurable, default `noreply@solo-ecommerce.com`

---

26. Security Infrastructure

Overview

Multi-layered security implementation covering authentication, authorization, input validation, and transport security.

Security Architecture (Defence in Depth)

```
Layer 1: NETWORK
├── TLS 1.2+ encryption in transit
├── CORS whitelist (frontend origin only)
└── Helmet HTTP security headers

Layer 2: RATE LIMITING
├── Global: 1000 requests/minute per IP
├── Auth endpoints: 5 per 15 minutes per email
└── API: 100 per minute per user

Layer 3: AUTHENTICATION
├── JWT access tokens (15-min expiry)
├── Refresh token rotation (7-day expiry)
└── Argon2id password hashing

Layer 4: AUTHORIZATION (RBAC)
├── Role: CUSTOMER — own resources only
├── Role: ADMIN — all resources + management
├── Role: SUPER_ADMIN — system settings
└── Guards: @Roles() decorator + RolesGuard

Layer 5: INPUT VALIDATION
├── class-validator DTOs on all endpoints
├── Whitelist: strip unknown properties
├── Transform: auto-type conversion
└── Parameterized queries (Prisma — no raw SQL injection)

Layer 6: DATA PROTECTION
├── Passwords: Argon2id (never stored plaintext)
├── Stripe keys: encrypted at rest
├── JWT secrets: environment variables only
├── Sensitive fields: excluded from responses (costPrice, password_hash)
```

Guards & Decorators

Guard/Decorator	Purpose
`@UseGuards(JwtAuthGuard)`	Require valid access token
`@UseGuards(RolesGuard)`	Check user role against required roles
`@Roles('ADMIN')`	Declare required role(s) for endpoint
`@Public()`	Skip authentication for endpoint
`@GetUser()`	Extract authenticated user from request

CORS Configuration

```
{
  origin: process.env.CORS_ORIGIN || 'http://localhost:52391',
  credentials: true,
  methods: ['GET', 'POST', 'PATCH', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization']
}
```

Helmet Headers

```
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
Content-Security-Policy: default-src 'self'
Strict-Transport-Security: max-age=31536000; includeSubDomains
```

End of Technical Features Document